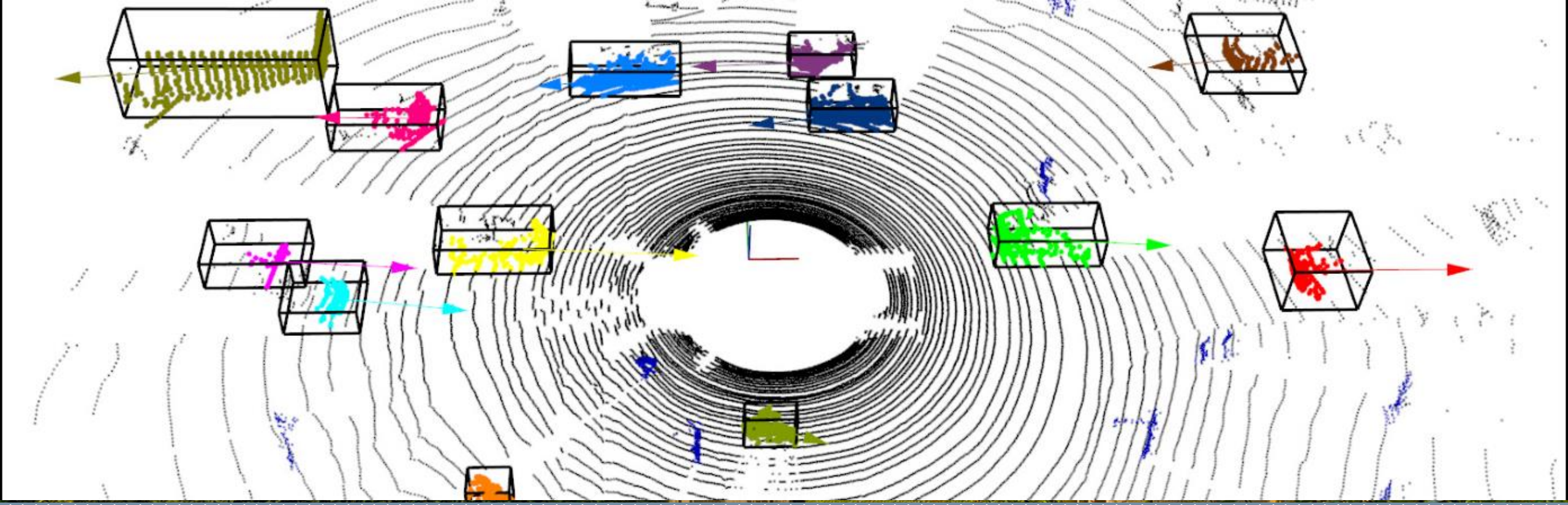
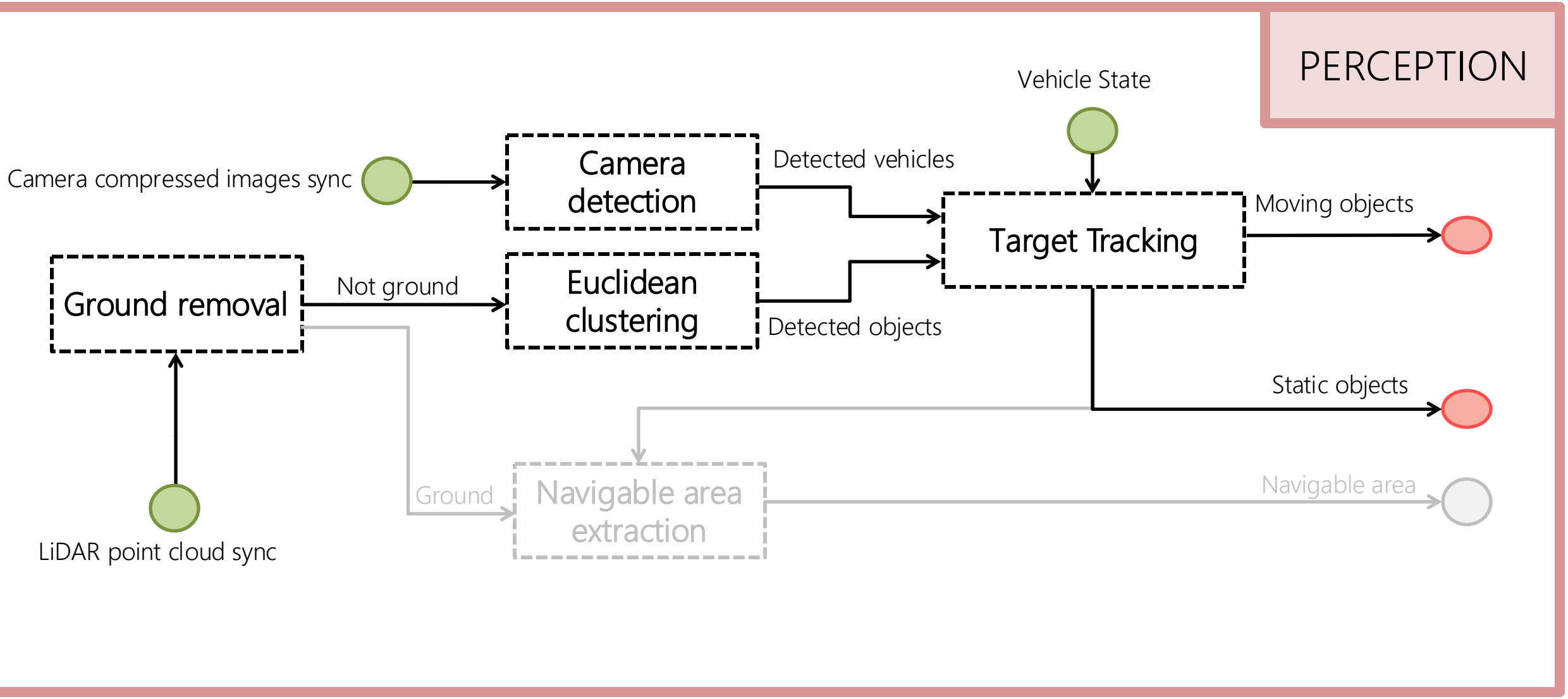


Automation and Control in Autonomous Vehicles (ACAV)

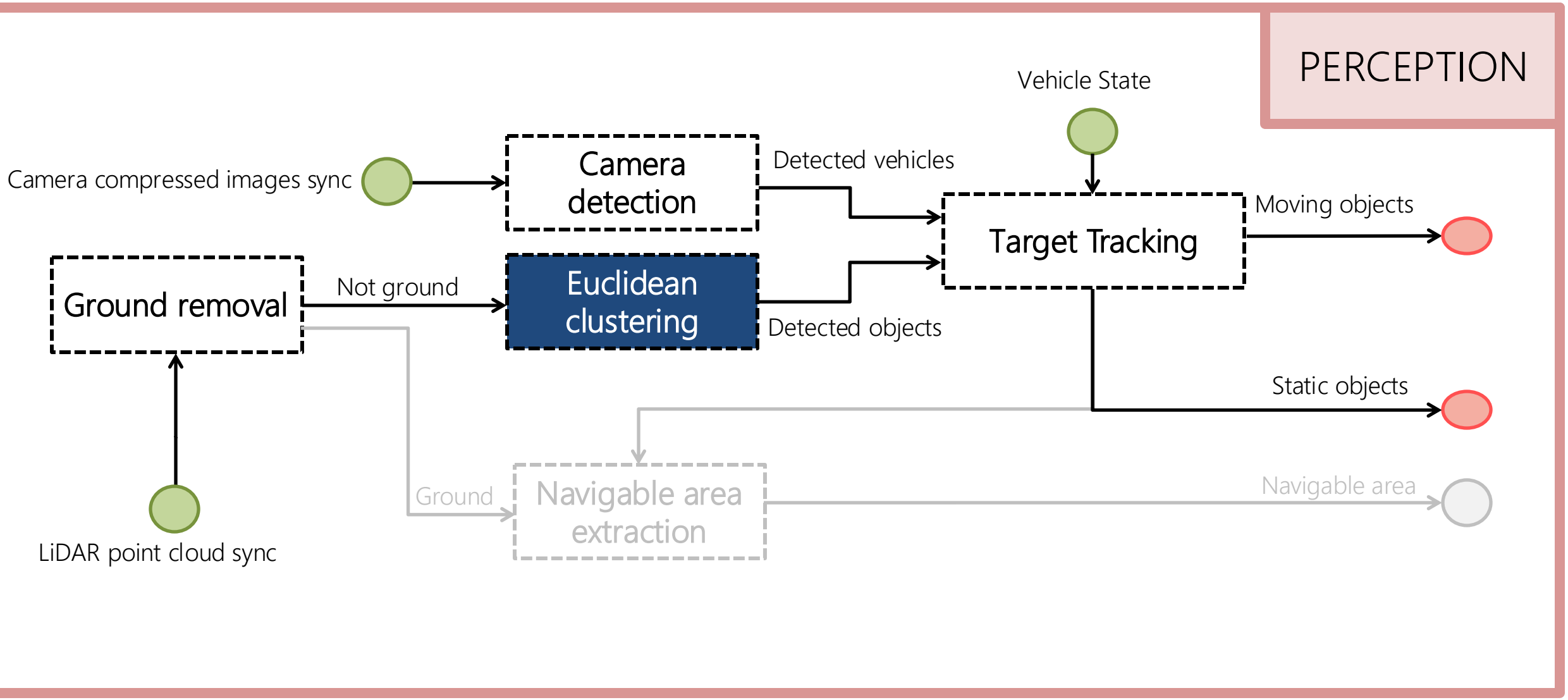
Target Tracking – Camera and LiDAR fusion

Matteo Corno

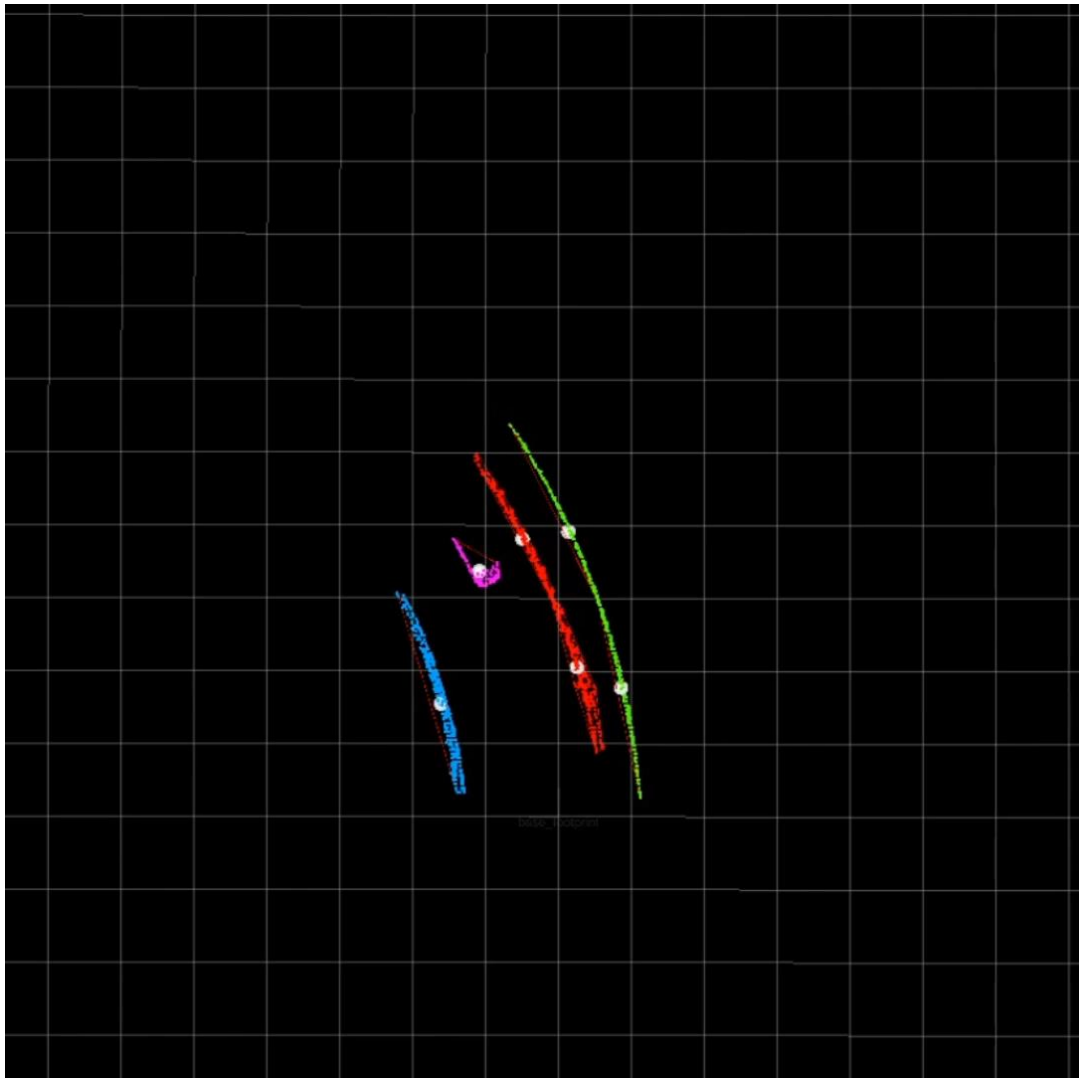
Perception Module



Perception Module



LiDAR Object Detection: Euclidean Clustering





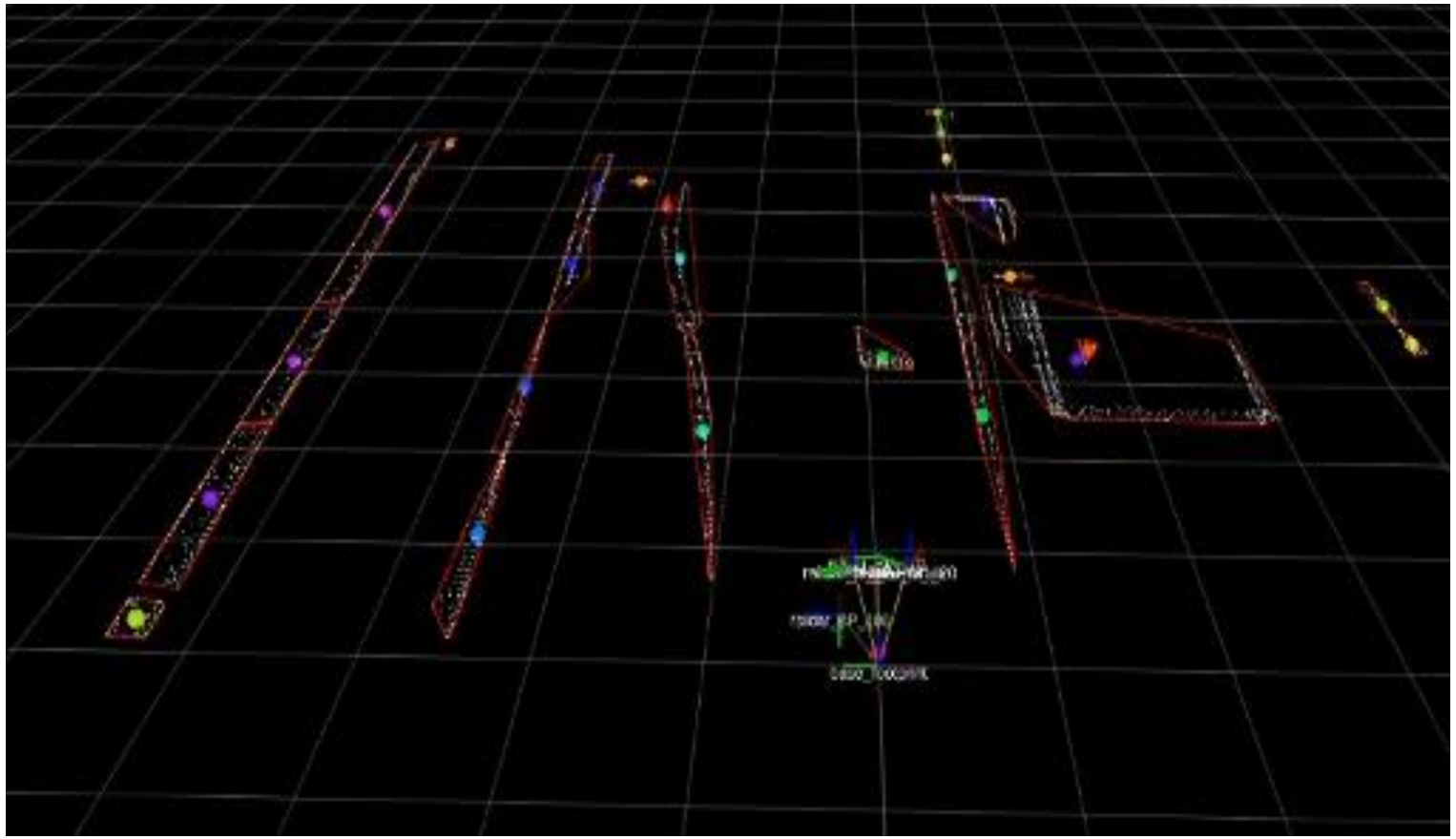
Algorithm
Euclidean Clustering

Input
Not ground points

Output
List of Detected Objects with:
- Centroid position (X, Y)
- Convex Hull Polygon

(Vehicle ref. frame)

LiDAR Object Detection: Euclidean Clustering





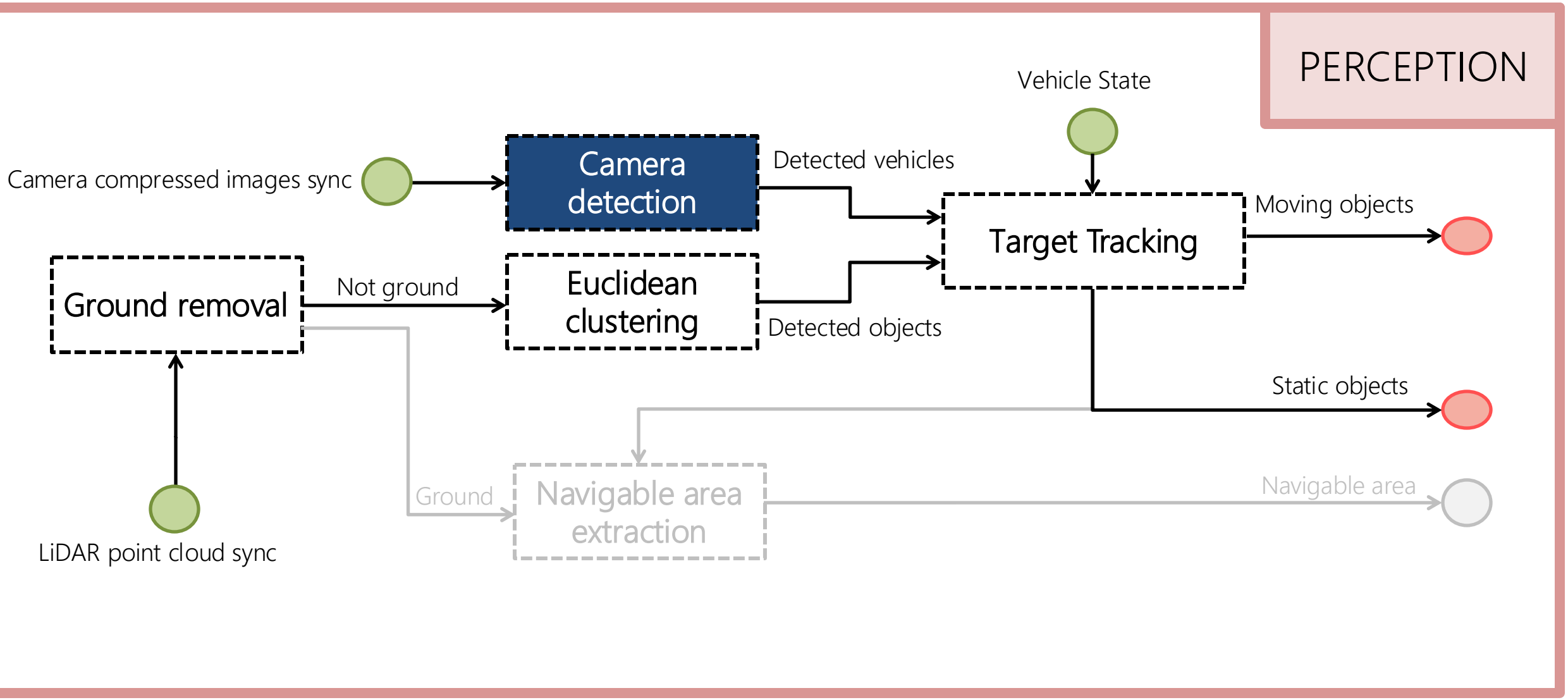
Algorithm
Euclidean Clustering (code)

Input
Not ground points

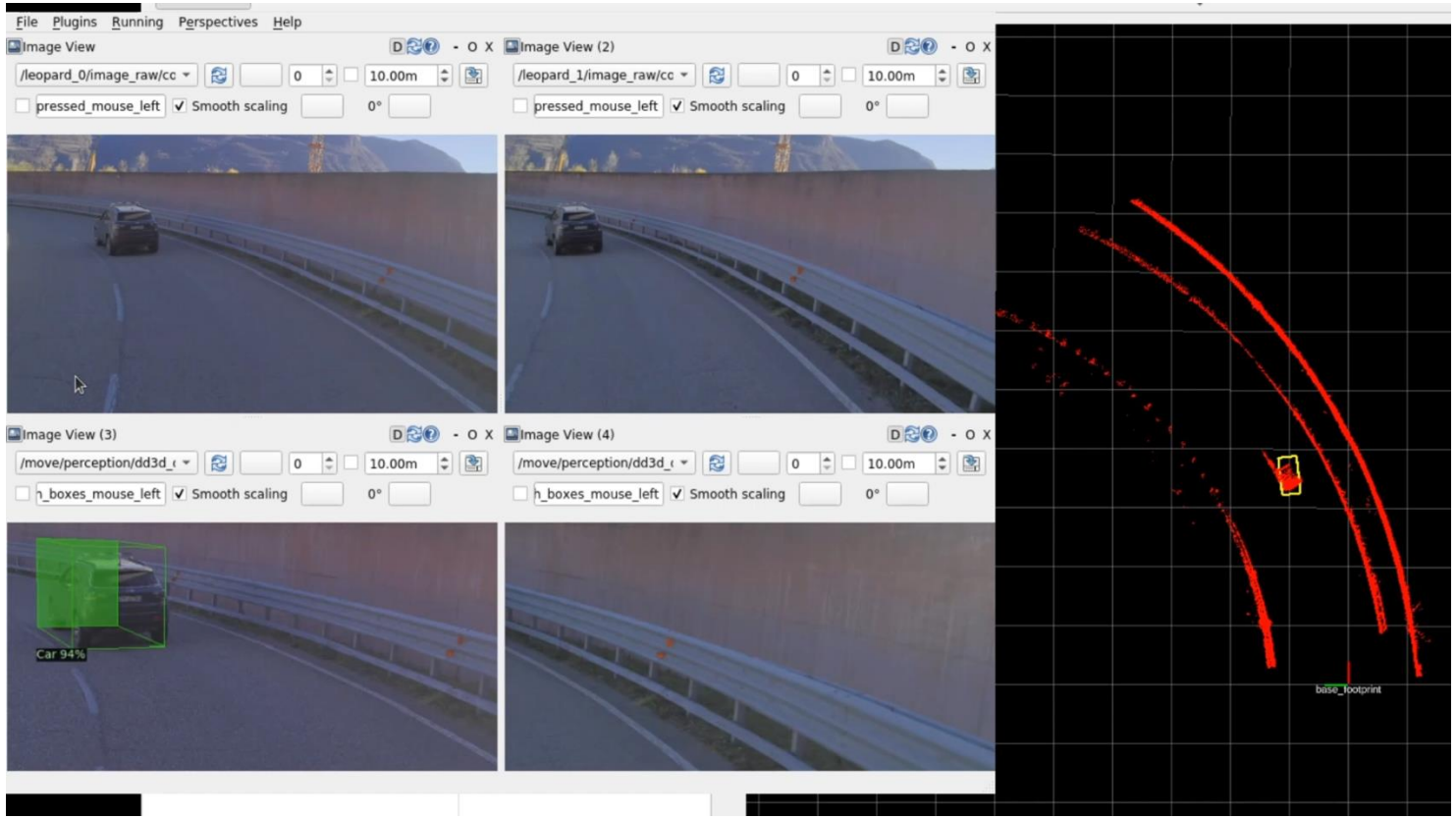
Output
List of Detected Objects with:
- Centroid position (X, Y)
- Convex Hull Polygon

(Vehicle ref. frame)

Perception Module



Camera Detection





Algorithm
DD3D (paper)

Input
Camera Image

Output
List of Detected Objects with:
- 3D Pose
- 2D and 3D Bounding Box

In vehicle reference frame

DD3D: Dense Depth-pre-trained 3D Detector

Why DD3D?
→ 3D position estimation from monocular cameras
→ Easier association with LiDAR data



Algorithm
DD3D (paper)

Input
Camera Image

Output
List of Detected Objects with:
- 3D Pose
- 2D and 3D Bounding Box

In vehicle reference frame

DD3D: Dense Depth-pre-trained 3D Detector

Why DD3D?
→ 3D position estimation from monocular cameras
→ Easier association with LiDAR data

Black-box usage: pre-trained on KITTI



Algorithm
DD3D (paper)

Input
Camera Image

Output
List of Detected Objects with:
- 3D Pose
- 2D and 3D Bounding Box

In vehicle reference frame

DD3D: Dense Depth-pre-trained 3D Detector

Why DD3D?
→ 3D position estimation from monocular cameras
→ Easier association with LiDAR data



Black-box usage: pre-trained on KITTI

Calibration parameters:
- Camera intrinsics → Rectify image and renders DD3D «camera-aware»
- Camera extrinsics → Transform boxes from camera to vehicle frame
- Image ROI → Process only part of the image

Algorithm
DD3D (paper)

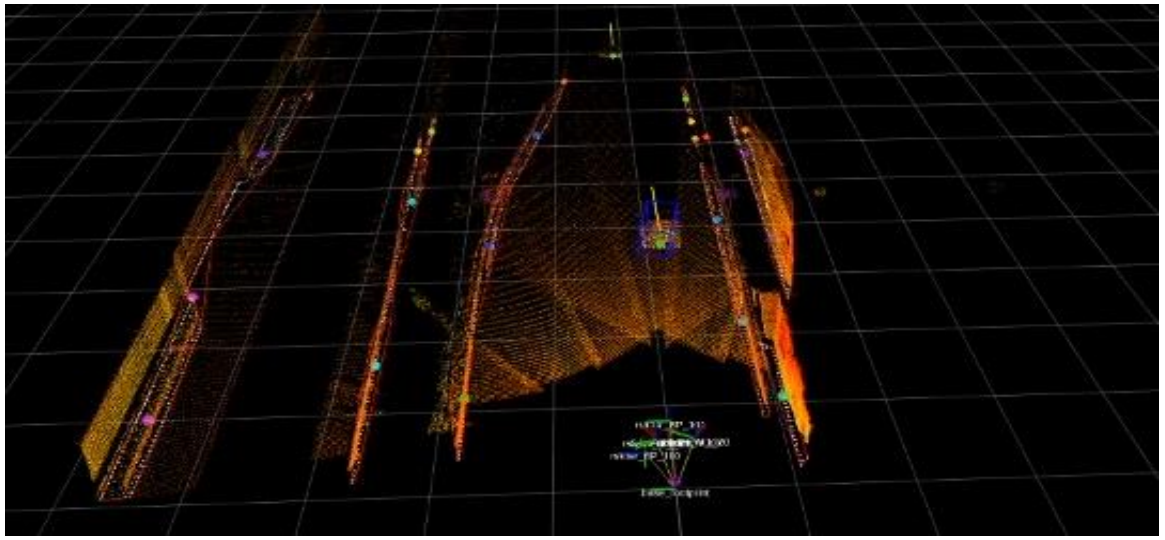
Input
Camera Image

Output
List of Detected Objects with:
- 3D Pose
- 2D and 3D Bounding Box

In vehicle reference frame

Camera Detection







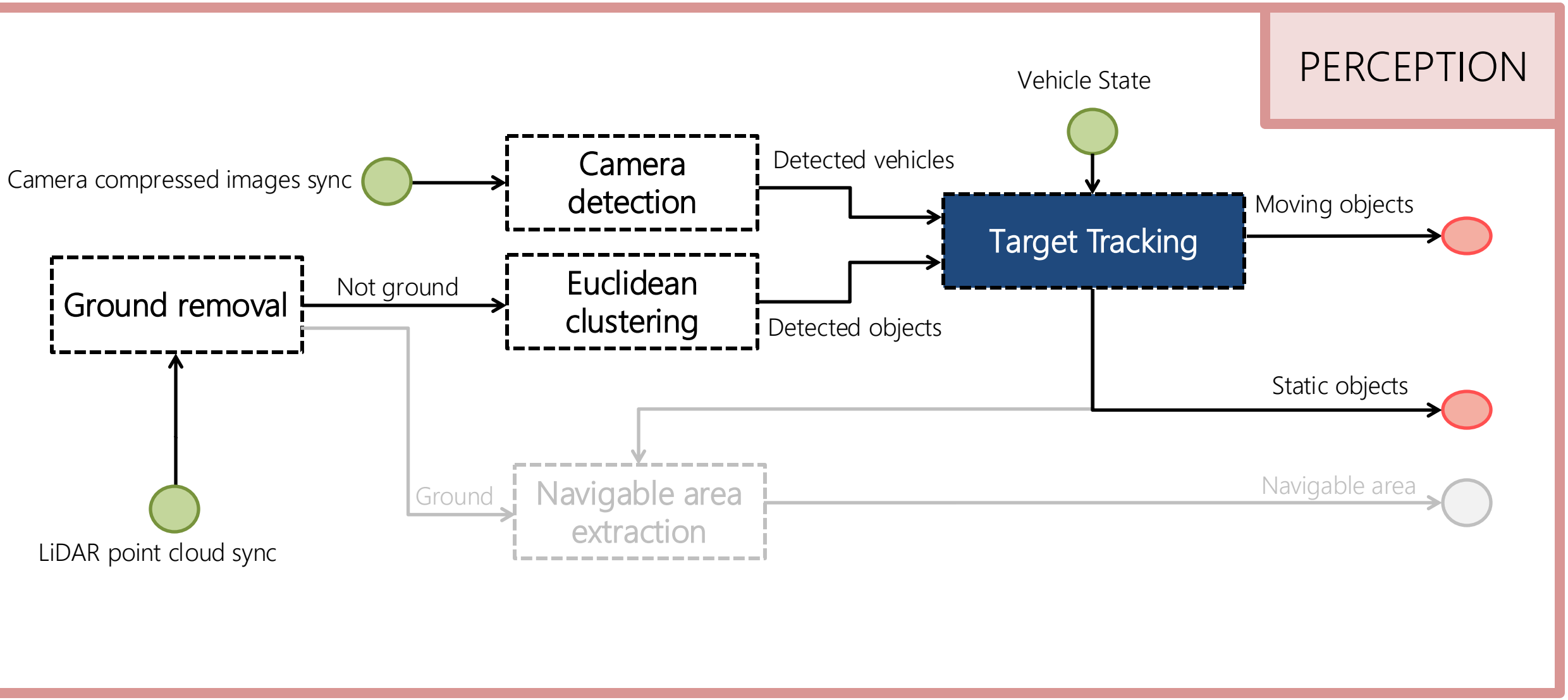
Algorithm
DD3D (paper)

Input
Camera Image

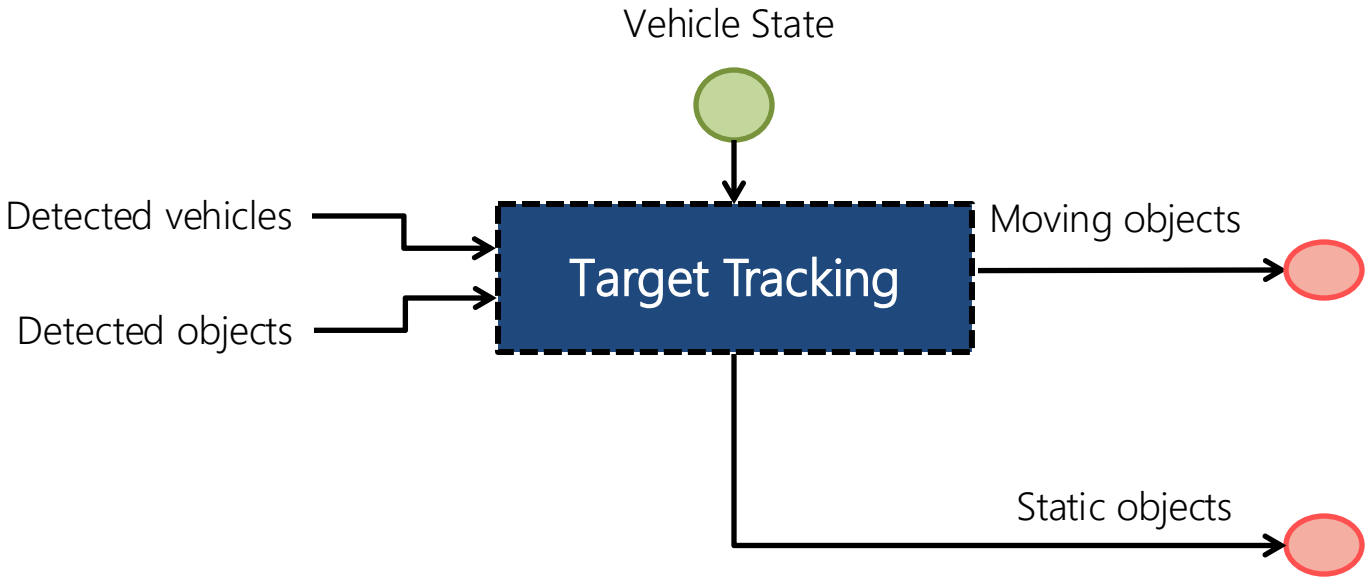
Output
List of Detected Objects with:
- 3D Pose
- 2D and 3D Bounding Box

In vehicle reference frame

Perception Module



Target Tracking



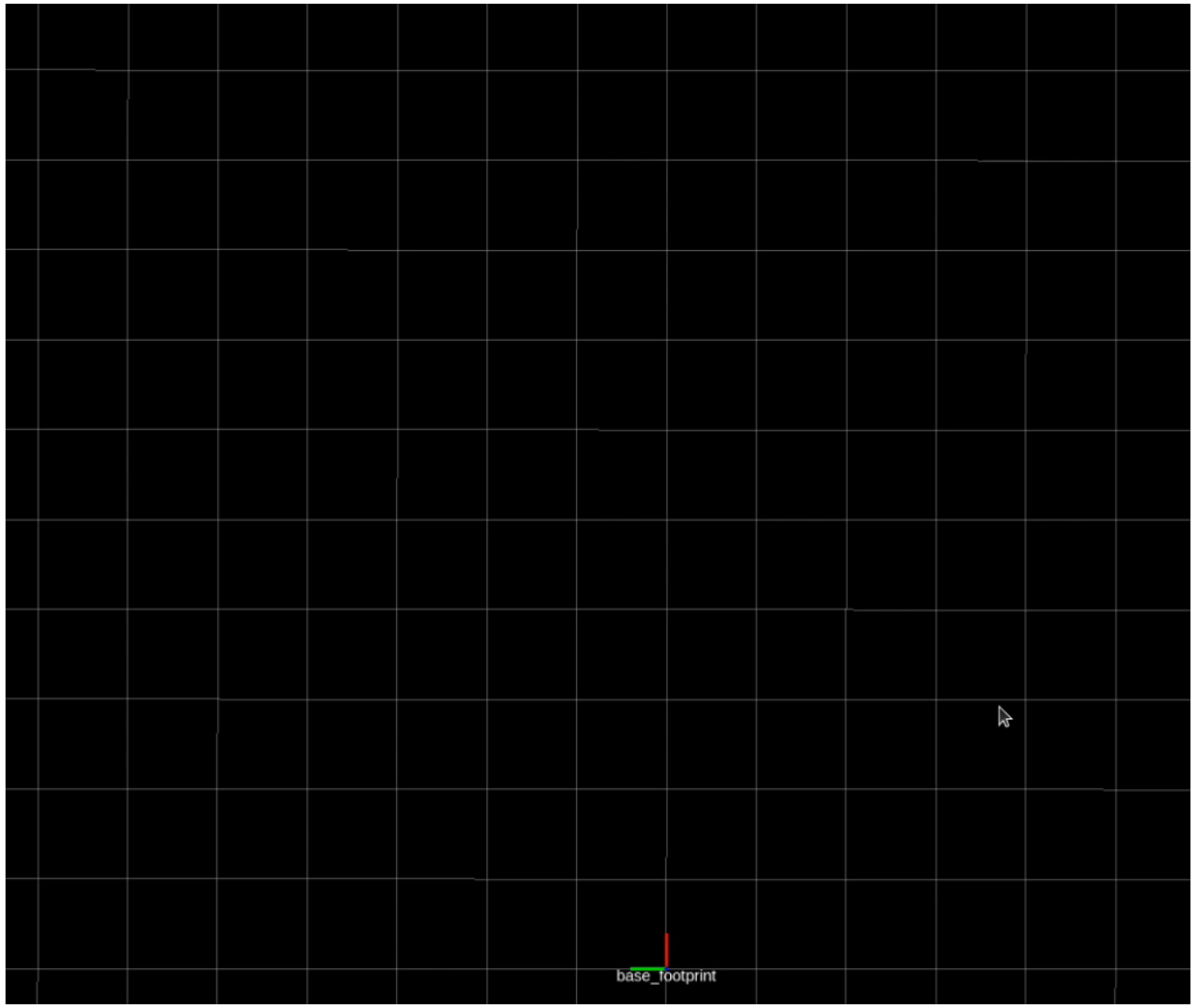
Goal
→ Distinguish between static and moving obstacles (vehicles)
→ Estimate speed of moving obstacles

Input
- LiDAR Detected Objects list:
  - 2D Centroid
  - 2D Convex Hull
- Camera Detected Objects list:
  - 3D Pose
  - 3D Bounding Box
- 2D Vehicle State (Pose, Velocity)

Output
- List of Moving Objects with:
  - 2D centroid pose (X, Y, ψ)
  - Linear, Angular Velocity
  - Convex Hull Polygon
- List of Static Objects with:
  - 2D centroid
  - 2D Convex Hull

In world-fixed (or odom) ref. frame

Target Tracking Output



Problem Defintion

estimation of tracks' dynamic states

→ multiple targets
→ at most one measurement per target per iteration (per sensor)

measurements from different sensors are processed independently

We built an EKF Multi Target Tracking algoritm performing track-level fusion of Camera and lidar measurements and M/N track management logic.

Finite State Machine (FSM) determines the birth and death of new and old tracks

Definitions: Measurements vs Tracks

Measurements

→ Unorderd, not consistent list
measure n is not always from object n

→ Partial state observability
LiDARs and cameras do not measure speed

→ Possibly false positives
objects wrongly detected as moving

→ Missed detections
Occlusions, asynchronous measures…

Tracks

→ Ordered, consistent
each track corresponds to a physical object

→ Full state estimation
Estimate object speed

→ False positives rejection

→ False negatives rejection
"The object is there even if you don't see it"

Each track is composed of:

EKF
Estimate and filter objects' states
$$X = [x \;\; y \;\; \psi \;\; v \;\; \omega]$$
(local reference frame)

Using clusters centroids as EKF measurements
$$Y = [x \;\; y]$$
(local reference frame)

Definitions: Measurements vs Tracks

Measurements

→ Unorderd, not consistent list
measure n is not always from object n

→ Partial state observability
LiDARs and cameras do not measure speed

→ Possibly false positives
objects wrongly detected as moving

→ Missed detections
Occlusions, asynchronous measures…

Tracks

→ Ordered, consistent
each track corresponds to a physical object

→ Full state estimation
Estimate object speed

→ False positives rejection

→ False negatives rejection
"The object is there even if you don't see it"

Each track is composed of:

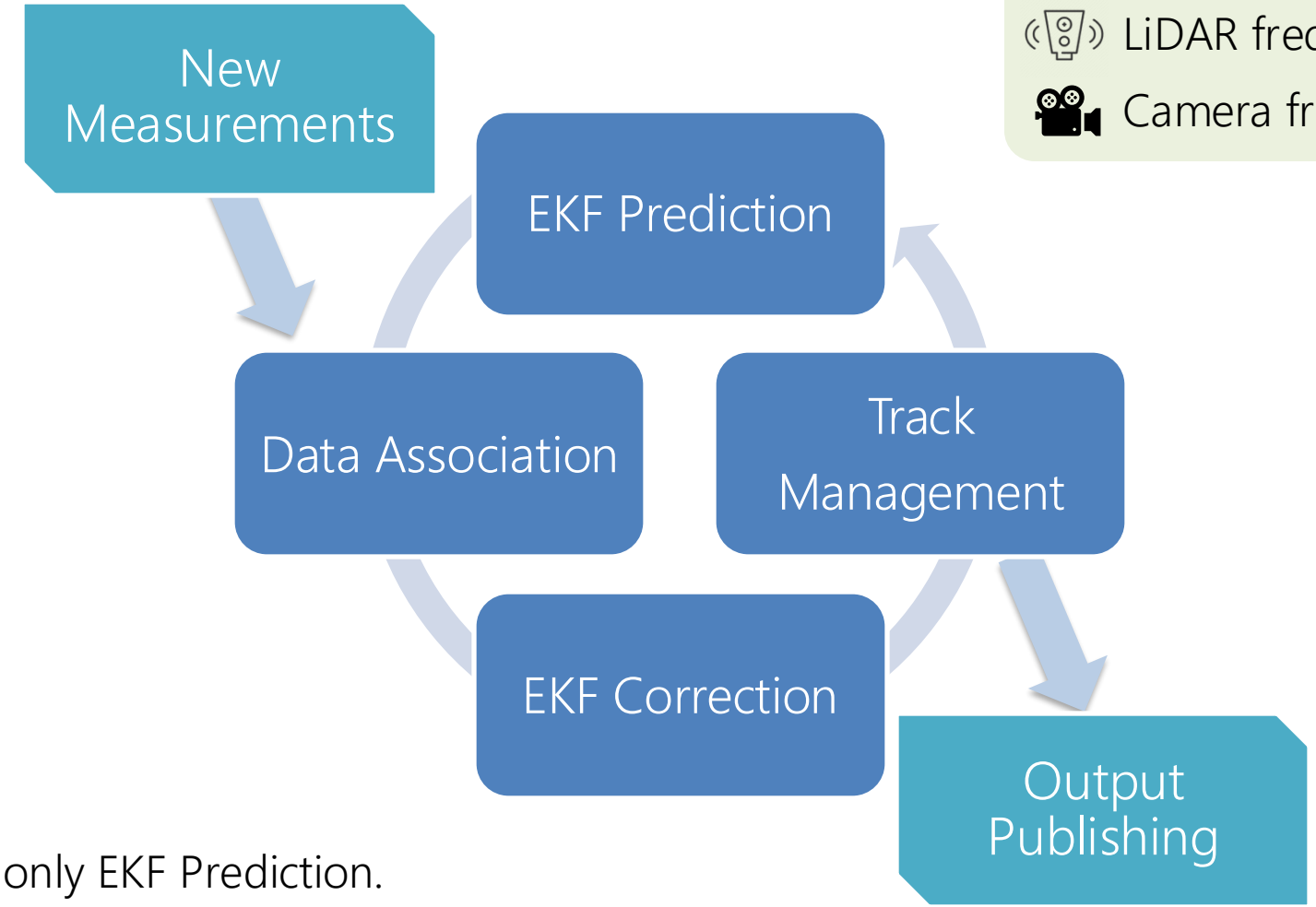
⇒ EKF

⇒ Finite State Machine
Manage track lifecycle

Target Tracking

Iterative algorithm composed of four steps.

- Fixed execution freq.: $33\ Hz$
- LiDAR freq. (not ground): $10\ Hz$
- Camera freq. (DD3D): $\approx 10\ Hz$

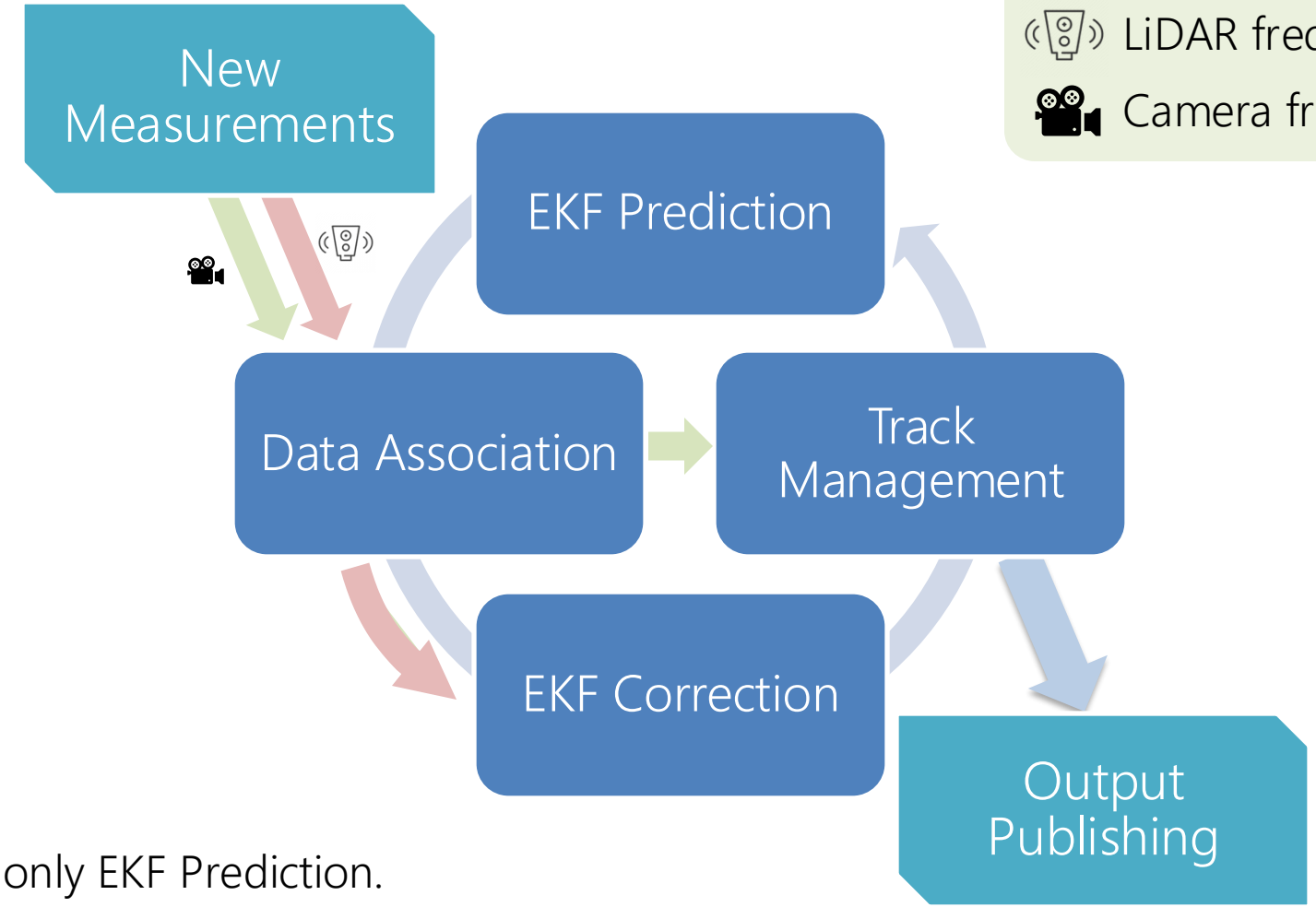


If no new measurements, only EKF Prediction.

Target Tracking

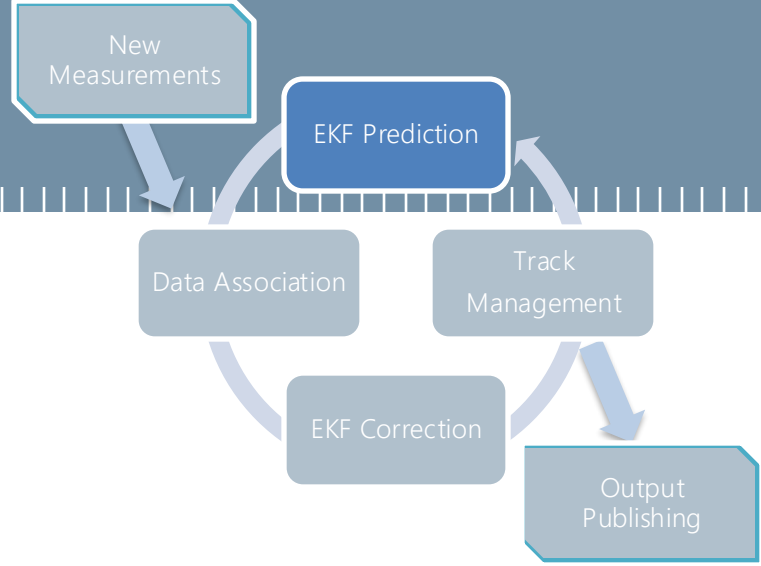
Iterative algorithm composed of four steps.

- Fixed execution freq.: $33\ Hz$
- LiDAR freq. (not ground): $10\ Hz$
- Camera freq. (DD3D): $\approx 10\ Hz$



If no new measurements, only EKF Prediction.

1 - EKF Prediction



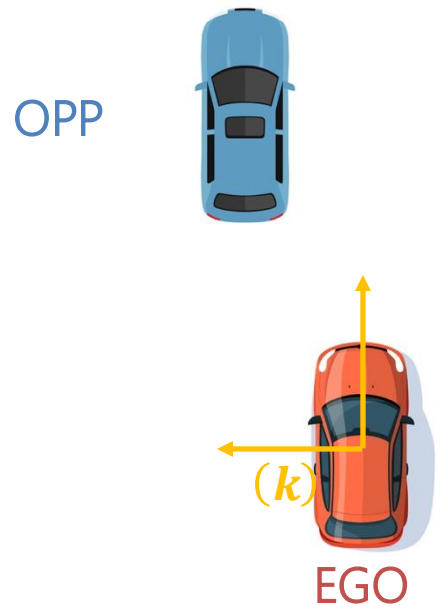
Idea
Based on last state, where should the tracked object be?
→ For each track, perform the EKF prediction step.

⇨ **EKF Model:** Coordinated turn motion (kinematic, local coords)

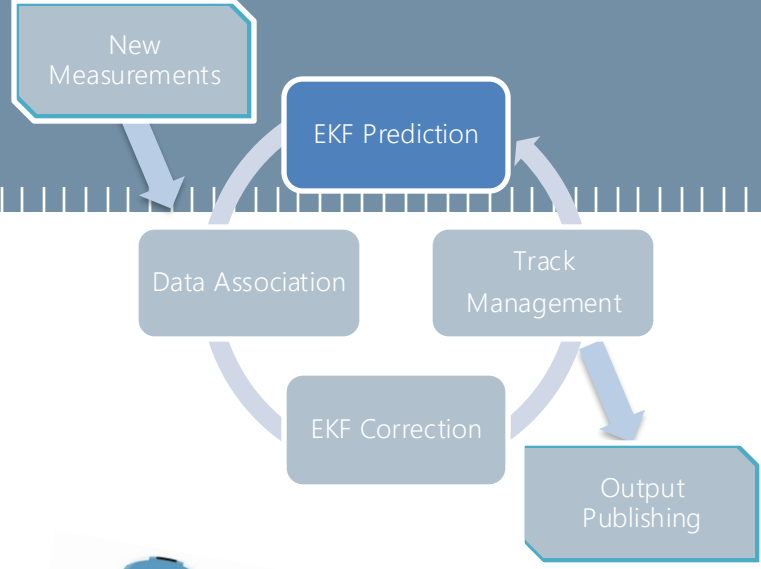
Assumptions:

- $\dot{v} = 0$ → constant speed
- $\dot{\omega} = 0$ → constant yaw rate

$$\begin{bmatrix} x_{k+1}^{(k)} \\ y_{k+1}^{(k)} \\ \psi_{k+1}^{(k)} \\ v_{k+1}^{(k)} \\ \omega_{k+1}^{(k)} \end{bmatrix} = \begin{bmatrix} x_k^{(k)} + \Delta T\, v_k^{(k)} \cos \psi_k^{(k)} \\ y_k^{(k)} + \Delta T\, v_k^{(k)} \sin \psi_k^{(k)} \\ \psi_k^{(k)} + \Delta T\, \omega_k^{(k)} \\ v_k^{(k)} \\ \omega_k^{(k)} \end{bmatrix}$$



1 - EKF Prediction



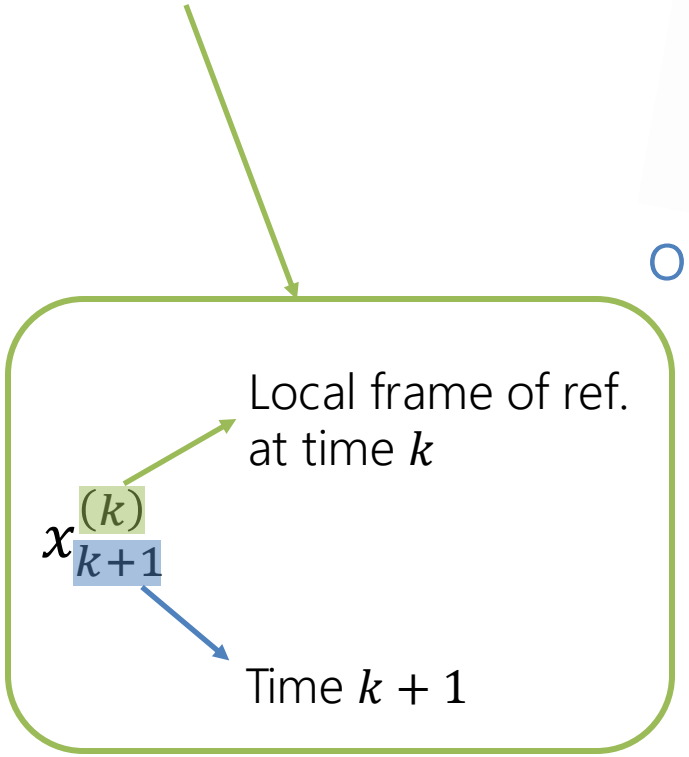
Idea
Based on last state, where should the tracked object be?
→ For each track, perform the EKF prediction step.

EKF Model: Coordinated turn motion (kinematic, local coords)

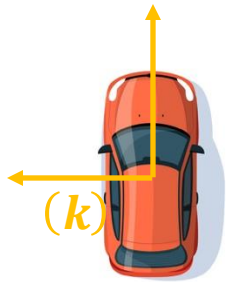
Assumptions:

- $\dot{v} = 0$ → constant speed
- $\dot{\omega} = 0$ → constant yaw rate

$$\begin{bmatrix} x_{k+1}^{(k)} \\ y_{k+1}^{(k)} \\ \psi_{k+1}^{(k)} \\ v_{k+1}^{(k)} \\ \omega_{k+1}^{(k)} \end{bmatrix} = \begin{bmatrix} x_k^{(k)} + \Delta T\, v_k^{(k)} \cos\psi_k^{(k)} \\ y_k^{(k)} + \Delta T\, v_k^{(k)} \sin\psi_k^{(k)} \\ \psi_k^{(k)} + \Delta T\, \omega_k^{(k)} \\ v_k^{(k)} \\ \omega_k^{(k)} \end{bmatrix}$$

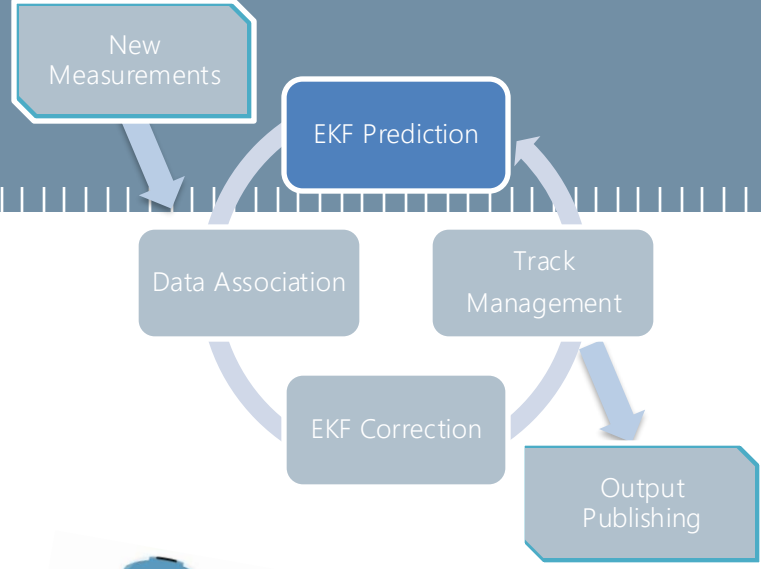



OPP



EGO

1 - EKF Prediction



Idea
Based on last state, where should the tracked object be?
→ For each track, perform the EKF prediction step.

EKF Model: Coordinated turn motion (kinematic, local coords)

Assumptions:

- $\dot{v} = 0$ → constant speed
- $\dot{\omega} = 0$ → constant yaw rate

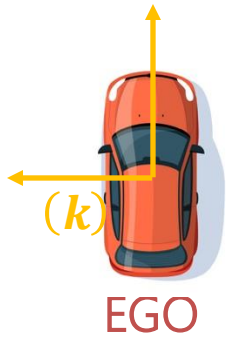


OPP

$$T_{(k)}^{(k+1)} = \begin{bmatrix} \cos(\Delta T \omega_{ego}) & -\sin(\Delta T \omega_{ego}) & \Delta T v_{x_{ego}} \\ \sin(\Delta T \omega_{ego}) & \cos(\Delta T \omega_{ego}) & \Delta T v_{y_{ego}} \\ 0 & 0 & 1 \end{bmatrix}$$

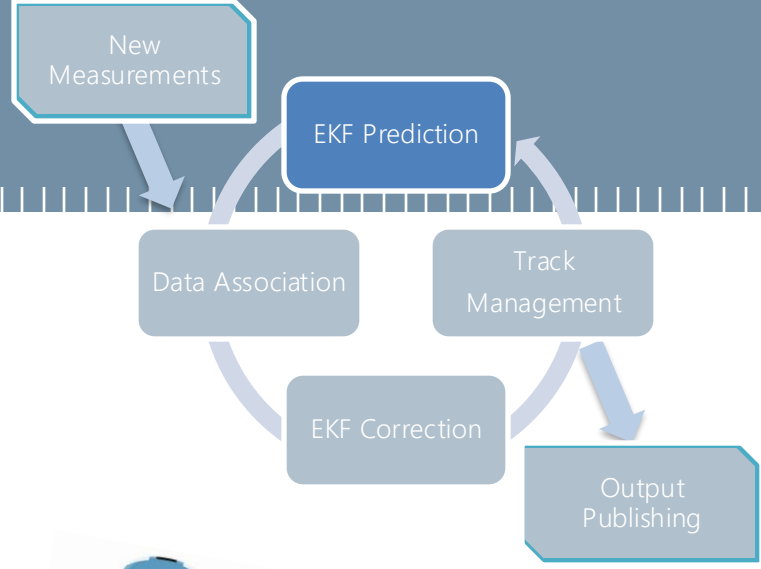
$$\begin{bmatrix} x_{k+1}^{(k)} \\ y_{k+1}^{(k)} \\ \psi_{k+1}^{(k)} \\ v_{k+1}^{(k)} \\ \omega_{k+1}^{(k)} \end{bmatrix} = \begin{bmatrix} x_k^{(k)} + \Delta T\, v_k^{(k)} \cos \psi_k^{(k)} \\ y_k^{(k)} + \Delta T\, v_k^{(k)} \sin \psi_k^{(k)} \\ \psi_k^{(k)} + \Delta T\, \omega_k^{(k)} \\ v_k^{(k)} \\ \omega_k^{(k)} \end{bmatrix}$$

$$\begin{bmatrix} x_{k+1}^{(k+1)} \\ y_{k+1}^{(k+1)} \\ \psi_{k+1}^{(k+1)} \\ v_{k+1}^{(k+1)} \\ \omega_{k+1}^{(k+1)} \end{bmatrix} = \begin{bmatrix} T_{(k+1)}^{(k)} \begin{bmatrix} x_{k+1}^{(k)} \\ y_{k+1}^{(k)} \\ \psi_{k+1}^{(k)} \end{bmatrix} \\ v_{k+1}^{(k)} \\ \omega_{k+1}^{(k)} \end{bmatrix}$$



EGO

1 - EKF Prediction



Idea
Based on last state, where should the tracked object be?
→ For each track, perform the EKF prediction step.

EKF Model: Coordinated turn motion (kinematic, local coords)

Assumptions:

- $\dot{v} = 0$ → constant speed
- $\dot{\omega} = 0$ → constant yaw rate

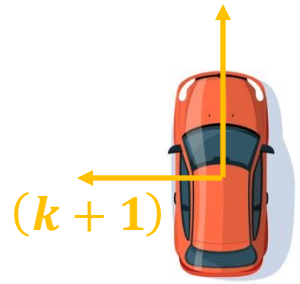


OPP

$$T_{(k)}^{(k+1)} = \begin{bmatrix} \cos(\Delta T \omega_{ego}) & -\sin(\Delta T \omega_{ego}) & \Delta T v_{x_{ego}} \\ \sin(\Delta T \omega_{ego}) & \cos(\Delta T \omega_{ego}) & \Delta T v_{y_{ego}} \\ 0 & 0 & 1 \end{bmatrix}$$

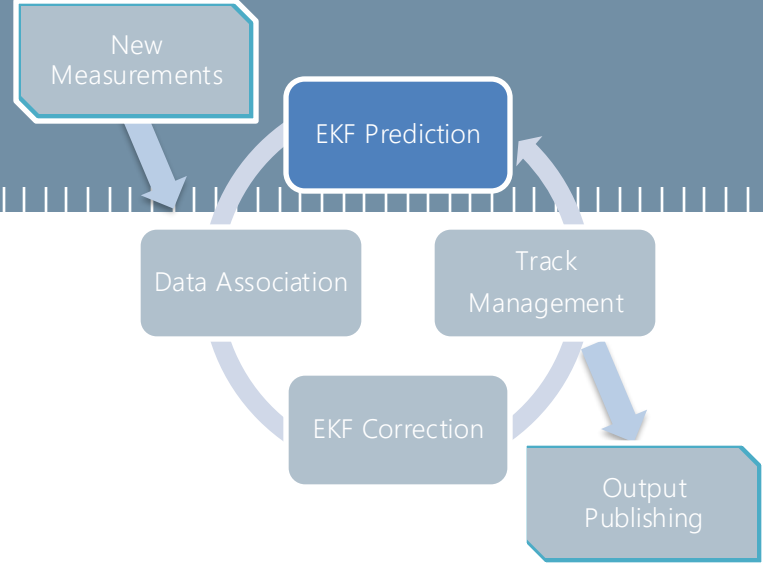
$$\begin{bmatrix} x_{k+1}^{(k)} \\ y_{k+1}^{(k)} \\ \psi_{k+1}^{(k)} \\ v_{k+1}^{(k)} \\ \omega_{k+1}^{(k)} \end{bmatrix} = \begin{bmatrix} x_k^{(k)} + \Delta T\, v_k^{(k)} \cos \psi_k^{(k)} \\ y_k^{(k)} + \Delta T\, v_k^{(k)} \sin \psi_k^{(k)} \\ \psi_k^{(k)} + \Delta T\, \omega_k^{(k)} \\ v_k^{(k)} \\ \omega_k^{(k)} \end{bmatrix}$$

$$\begin{bmatrix} x_{k+1}^{(k+1)} \\ y_{k+1}^{(k+1)} \\ \psi_{k+1}^{(k+1)} \\ v_{k+1}^{(k+1)} \\ \omega_{k+1}^{(k+1)} \end{bmatrix} = \begin{bmatrix} T_{(k+1)}^{(k)} \begin{bmatrix} x_{k+1}^{(k)} \\ y_{k+1}^{(k)} \\ \psi_{k+1}^{(k)} \end{bmatrix} \\ v_{k+1}^{(k)} \\ \omega_{k+1}^{(k)} \end{bmatrix}$$



EGO

1 - EKF Prediction – Prediction Equations



➢ EKF Prediction Equation:

$$\hat{\boldsymbol{X}}(\boldsymbol{t}) = \boldsymbol{f}(\boldsymbol{X}(\boldsymbol{t}-\boldsymbol{1}))$$
$$\hat{\boldsymbol{Y}}(\boldsymbol{t}) = \boldsymbol{H}\hat{\boldsymbol{X}}(\boldsymbol{t})$$
$$\boldsymbol{P}(\boldsymbol{t}) = \boldsymbol{A}(\boldsymbol{t})\boldsymbol{P}(\boldsymbol{t}-\boldsymbol{1})\boldsymbol{A}(\boldsymbol{t})' + \boldsymbol{Q}$$
$$\boldsymbol{S}(\boldsymbol{t}) = \boldsymbol{H}\boldsymbol{P}(\boldsymbol{t})\boldsymbol{H}' + \boldsymbol{R}$$

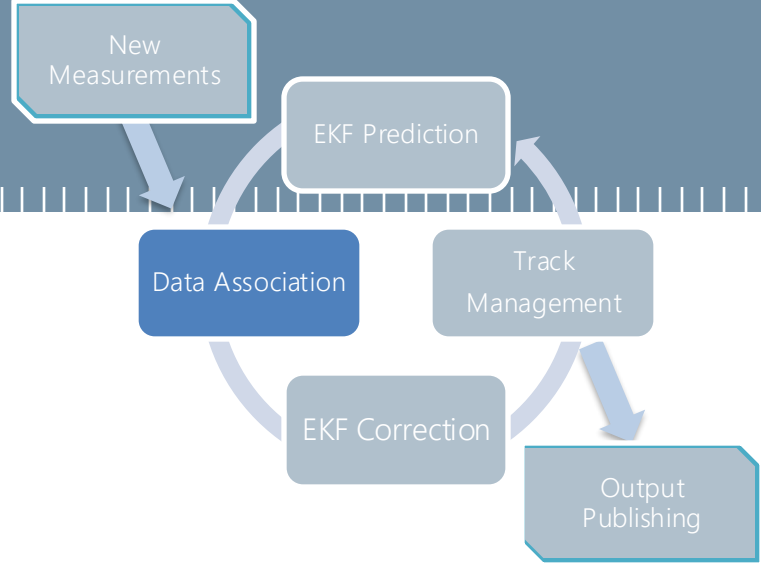
Where:

- $\hat{\boldsymbol{Y}}\,(t)$ are the predicted measurements in EGO r.f.
- H is the output matrix
- $P(t)$ is the current estimate of the state error covariance
- $A(t)$ is the linearization of the state equations around $X(t)$
- Q is the model error covariance
- R is the measurement error covariance
- $\boldsymbol{S}(\boldsymbol{t})$ is the current estimate of the measurement error covariance

$$H = \begin{bmatrix} 1 & 0 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 & 0 \end{bmatrix}$$

$$A_{i,j} = \frac{dX_i}{dX_j}$$

2 - Data Association



Idea
We have N measurements and M tracks.
Which measurement is referred to which track?
→ Find associations by minimizing overall measure-track distances

Measures

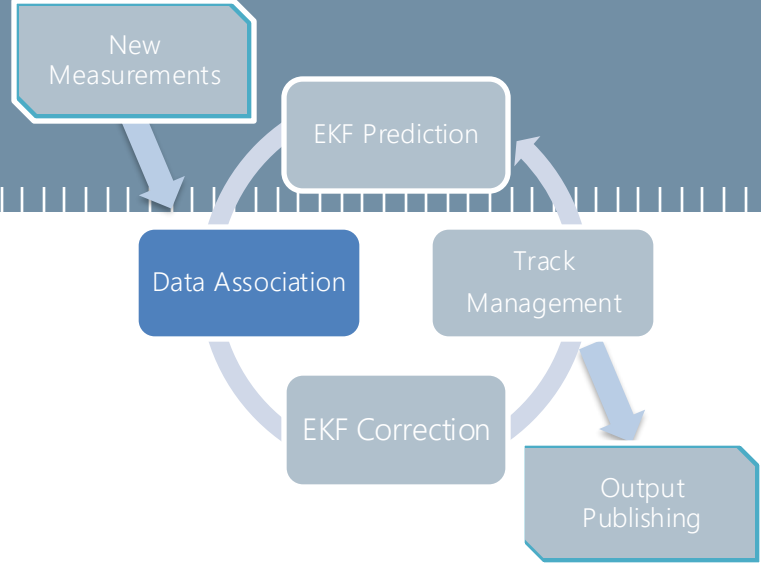
Tracks	0	1
0	4	0,05
1	0,04	5

Munkres
aka
Hungarian algorithm

Measures

Tracks	0	1
0	4	0,05 ✓
1	0,04 ✓	5

2 - Data Association



Idea
We have N measurements and M tracks.
Which measurement is referred to which track?
→ Find associations by minimizing overall measure-track distances

Measures

Tracks	0	1
0	4	0,05
1	0,04	5

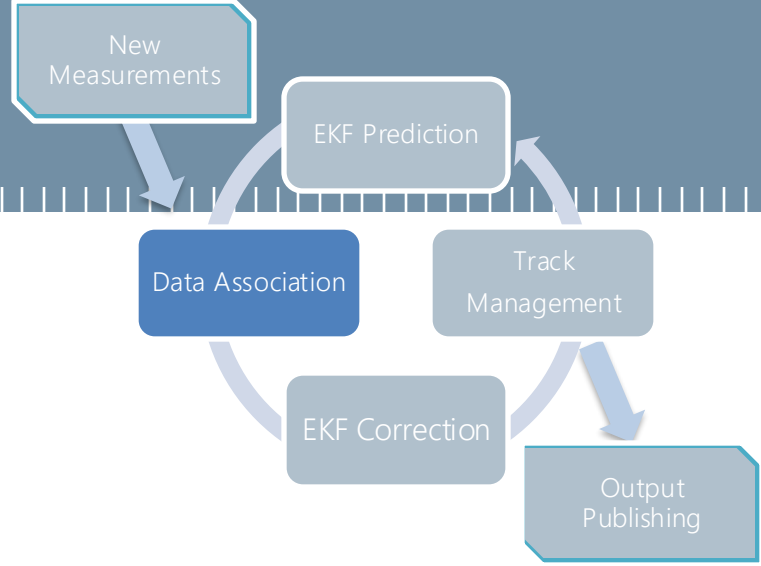
Munkres
aka
Hungarian algorithm

Measures

Tracks	0	1
0	4	0,05 ✓
1	0,04 ✓	5

How to build the Measures-Tracks distance table?

2 - Data Association



Euclidean distance does not take into account uncertainties

↓

Mahalanobis distance $D_M(Y, \hat{Y})$

$$D_M(Y, \hat{Y}) = \sqrt{(Y - \hat{Y})^T S^{-1} (Y - \hat{Y})}$$

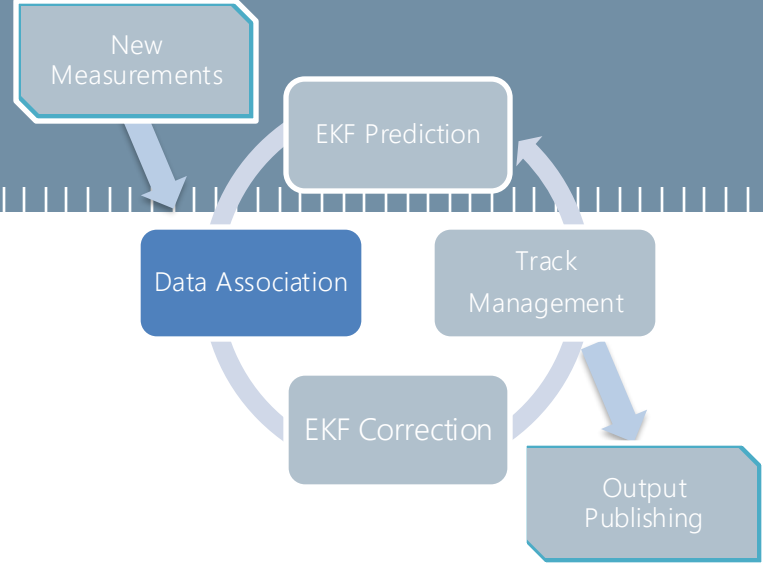
Error between predicted position and measured position

Innovation covariance

$$S(t) = HP(t)H^T + R$$

Contains an estimate of both state prediction and measurement

2 - Data Association - Gating



Idea
Discard statistically incompatible associations
→ Impose a threshold over the maximum distance
→ Allowing some measures to go unassociated

$$\begin{matrix} Y \sim \mathcal{N} \\ \hat{Y} \sim \mathcal{N} \end{matrix} \Rightarrow D_M \sim \chi^2(2) \longrightarrow$$ We can set a threshold based on a confidence level

Measures

Tracks	0	1
0	15	0,05
1	9	5

Gating Threshold

Measures

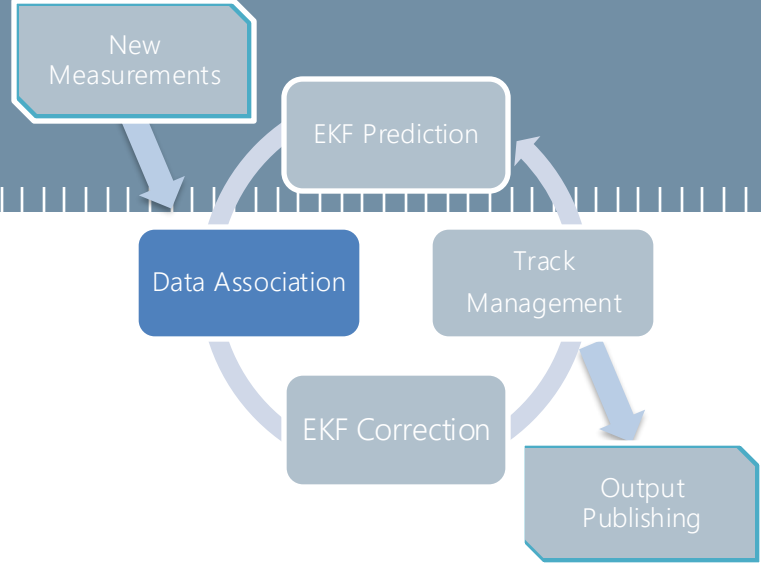
Tracks	0	1
0	~~15~~	0,05
1	~~9~~	5

Munkres

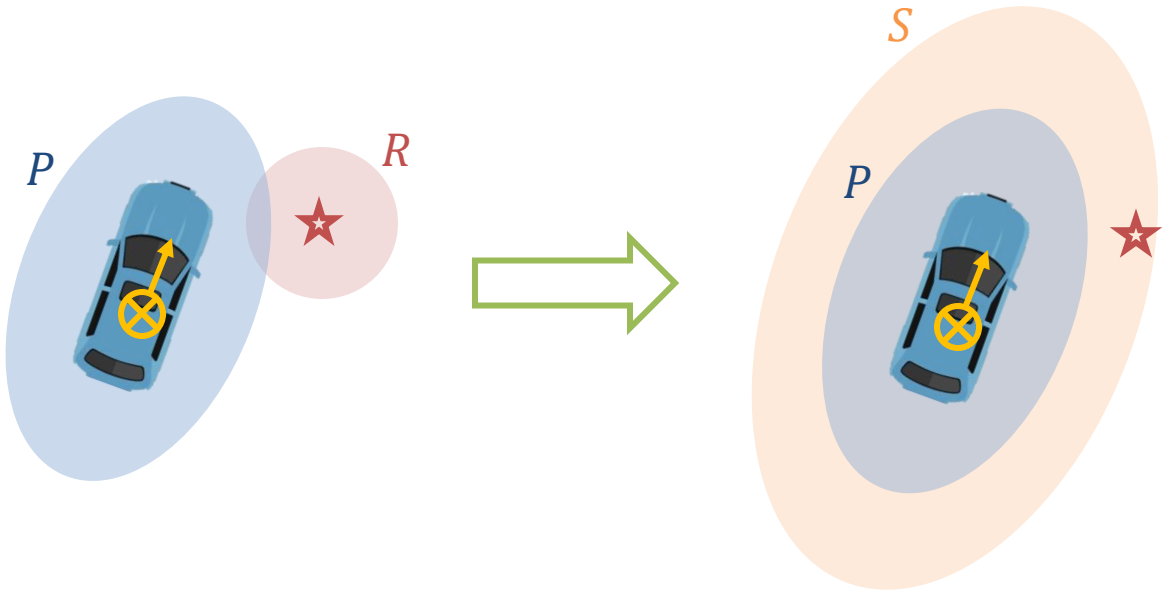
Measures

Tracks	0	1
0	~~15~~	0,05
1	~~9~~	5

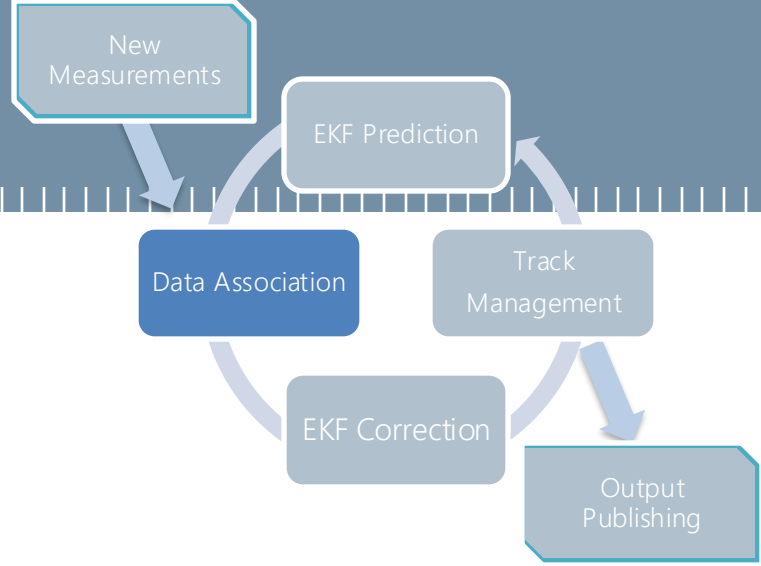
2 - Data Association - Gating



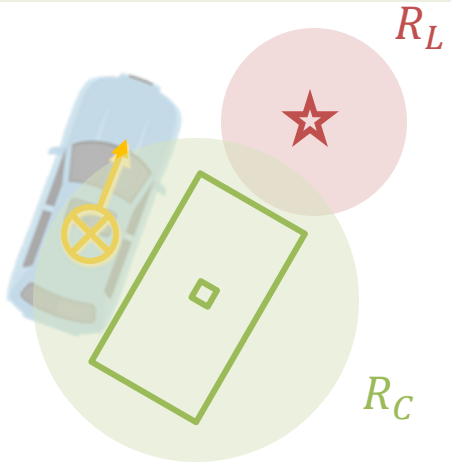
Idea
Discard statistically incompatible associations
→ Impose a threshold over the maximum distance
→ Allowing some measures to go unassociated



2 - Data Association – Cameras



Idea
Validate LiDAR measurements with camera detections
→ Camera – LiDAR association
→ Same workflow as LiDAR – track association



LiDAR \ Camera	0	1
0	15	0,05
1	9	5

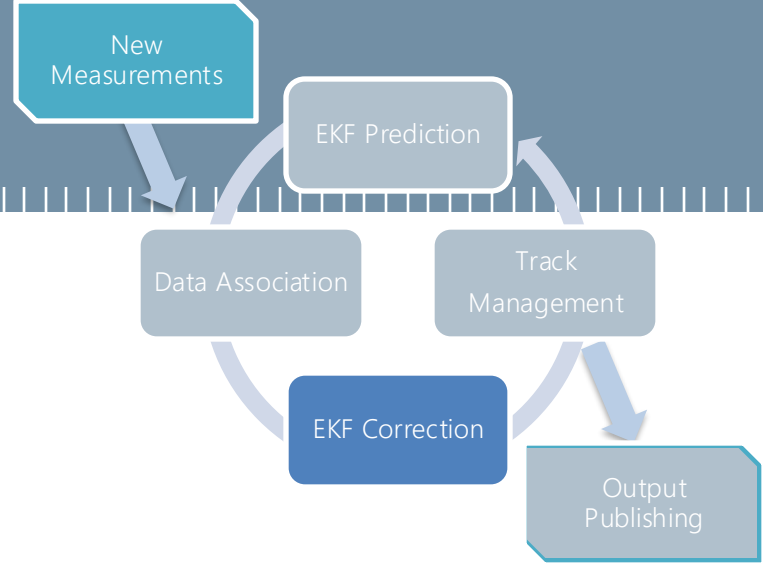
Gating Threshold

LiDAR \ Camera	0	1
0	~~15~~	0,05
1	~~9~~	5

Munkres

LiDAR \ Camera	0	1
0	~~15~~	0,05
1	~~9~~	5

3 – EKF Correction



- For all tracks that have a measure associated, perform **EKF correction:**

1. Compute the **error** (done in previous step):

$$E(t) = Y(t) - \hat{Y}(t) = Y(t) - H\hat{X}(t)$$

2. Compute the **Kalman Gain:**

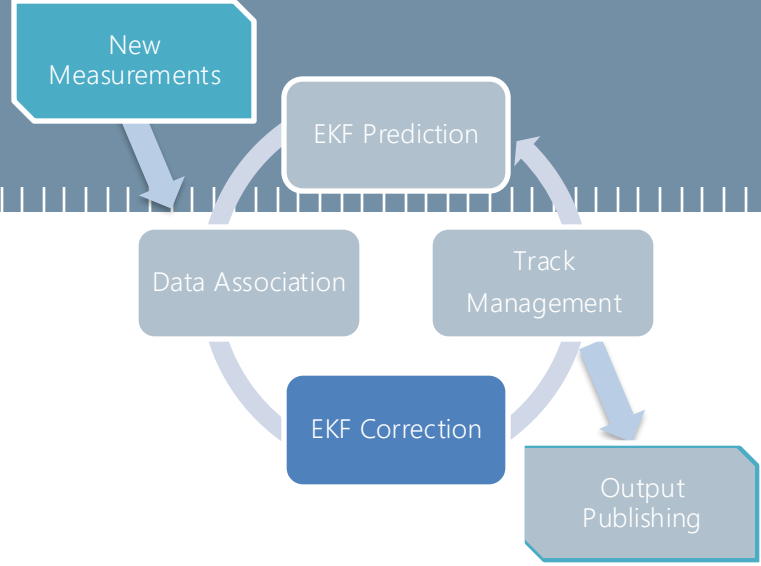
$$L(t) = P(t)HS(t)^{-1}$$

3. Update the **state estimation**:

$$X(t) = \hat{X}(t) + L(t) * E(t)$$



3 – EKF Correction



- For all tracks that have a measure associated, perform **EKF correction:**

1. Compute the **error** (done in previous step):

$$E(t) = Y(t) - \hat{Y}(t) = Y(t) - H\hat{X}(t)$$

2. Compute the **Kalman Gain:**

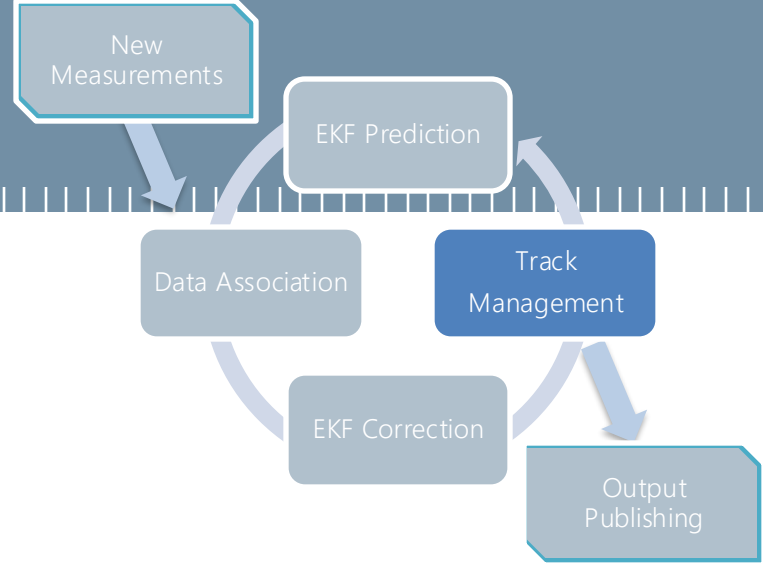
$$L(t) = P(t)HS(t)^{-1}$$

3. Update the **state estimation**:

$$X(t) = \hat{X}(t) + L(t) * E(t)$$

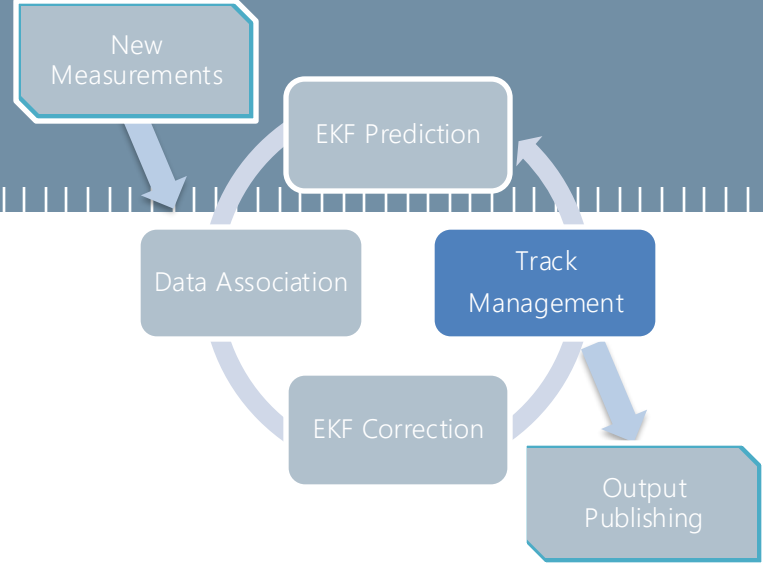


4 – Track Management



- Track Management logic is a **Finite State Machine (FSM)** where each state transition is determined by the **M/N** logic, aka:
  - «Have I received more/less than N measurements in the last M iterations?»
- This logic is implemented in parallel for LiDAR and Camera so we have:
  - N_{LiDAR} = number of LiDAR associated measures in last M iterations
  - N_{Cam} = number of Camera associated measures in last M iterations
  - M = State change threshold. Can be M_{accept}, M_{conf} or M_{elim}
  - $\#_{it}$ = number of iterations since track birth

4 – Track Management FSM



$N_{LiDAR} = 1$
$AND\ \ N_{Cam} = 1$

TENTATIVE

$N_{LiDAR} > M_{accept}$
$OR\ N_{Cam} > M_{accept}$

ACCEPTED

$N_{LiDAR} > M_{conf}$
$AND\ N_{Cam} > M_{conf}$

CONFIRMED

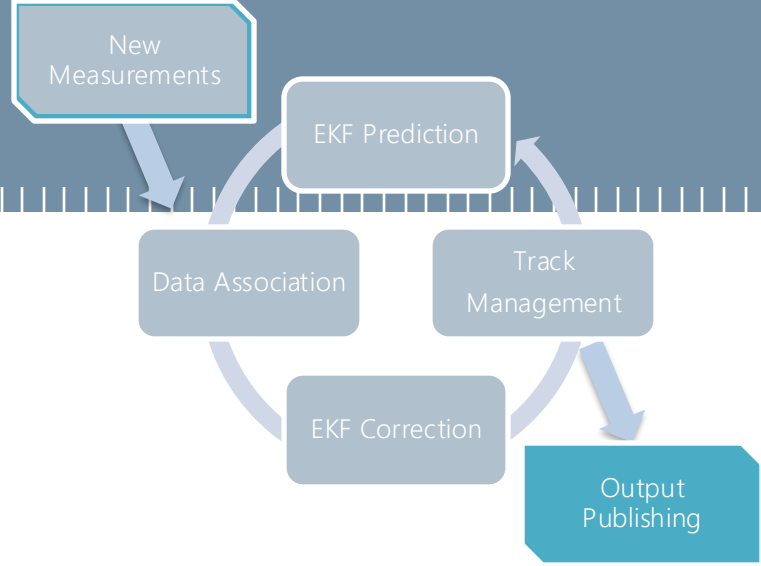
$N_{LiDAR} < M_{elim}$
$OR\ N_{Cam} < M_{elim}$

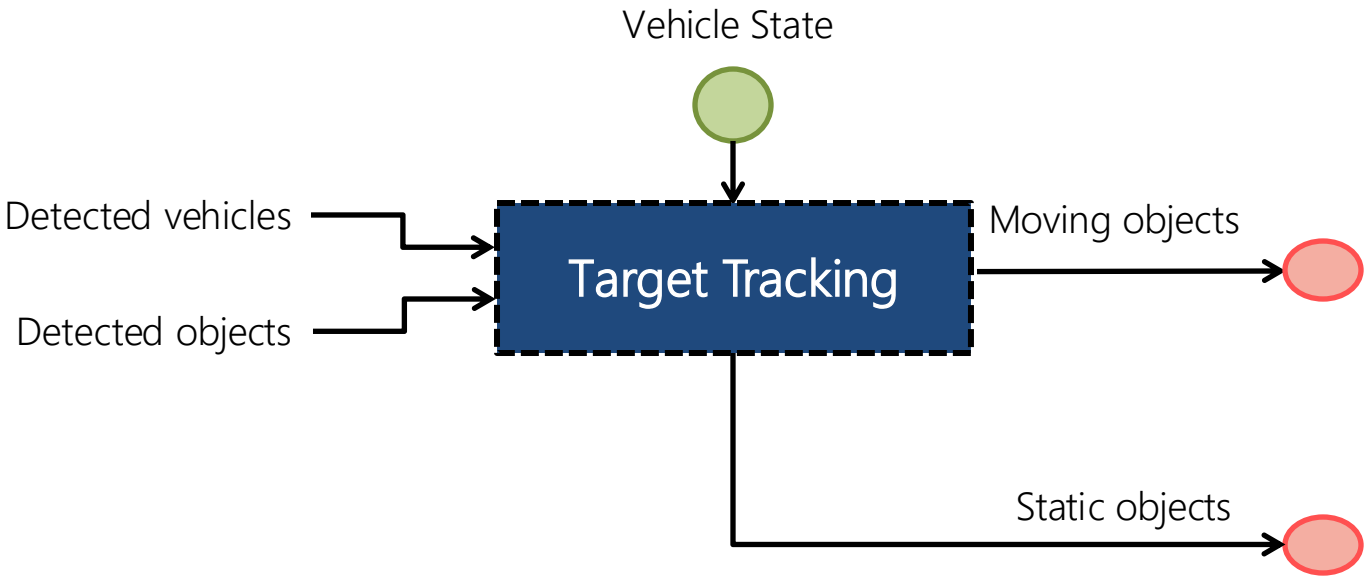
$\#_{it} = M$

DEAD

$N_{LiDAR} < M_{elim}$
$OR\ N_{Cam} < M_{elim}$

5 – Output Composition

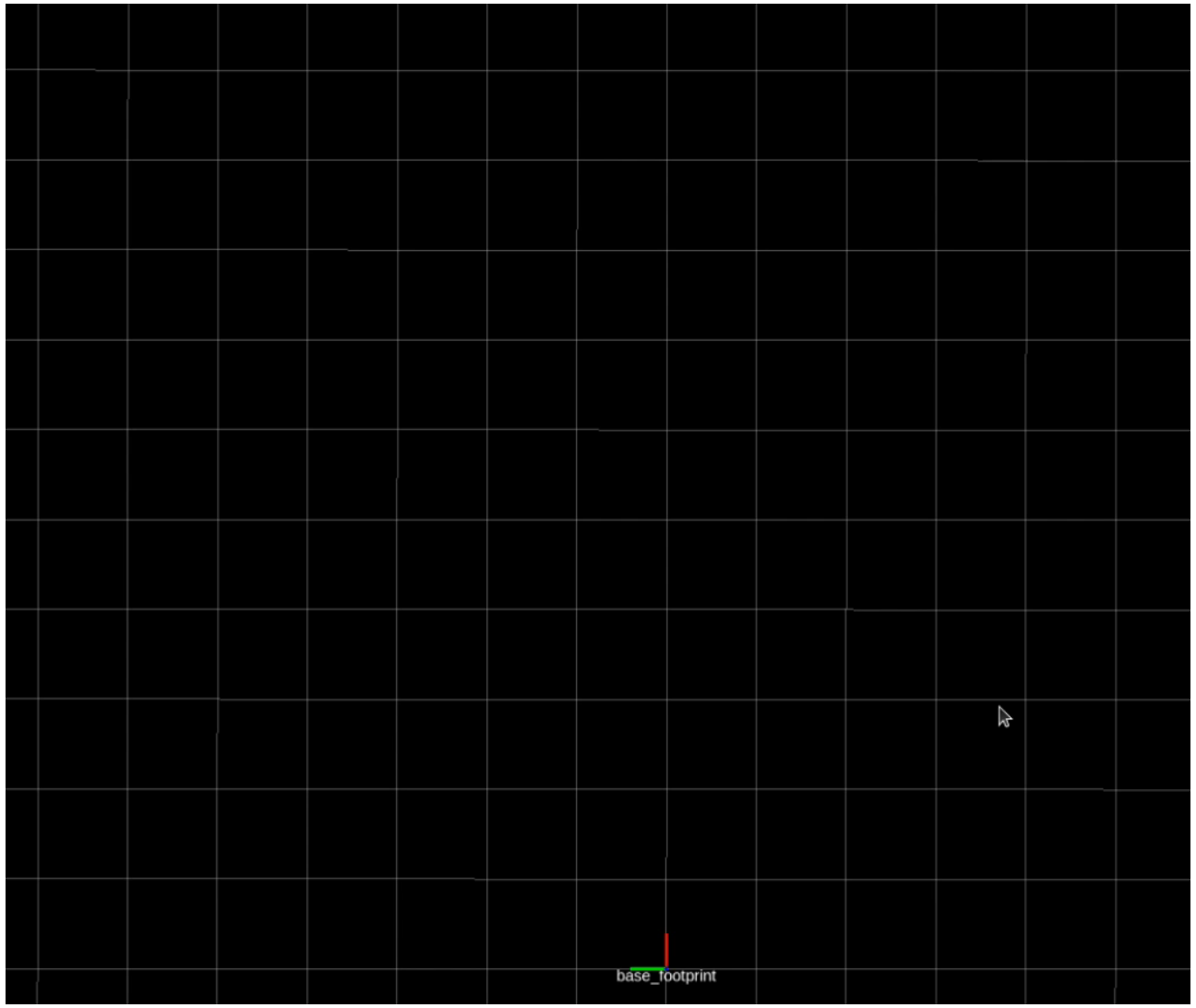




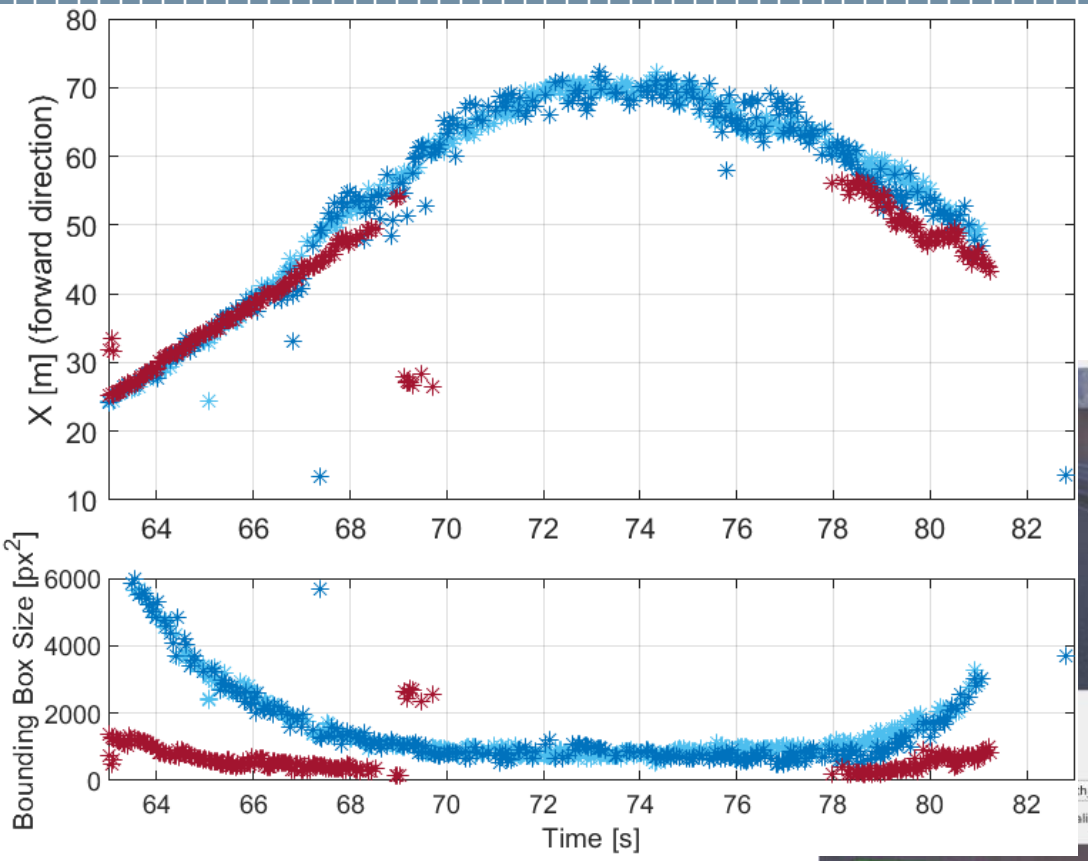
Confirmed tracks
- Clusters (centroid+vertices)
- Estimated speed and heading

Unassociated LiDAR measures
(+ safety check on min track distance)
- Clusters (centroid+vertices)

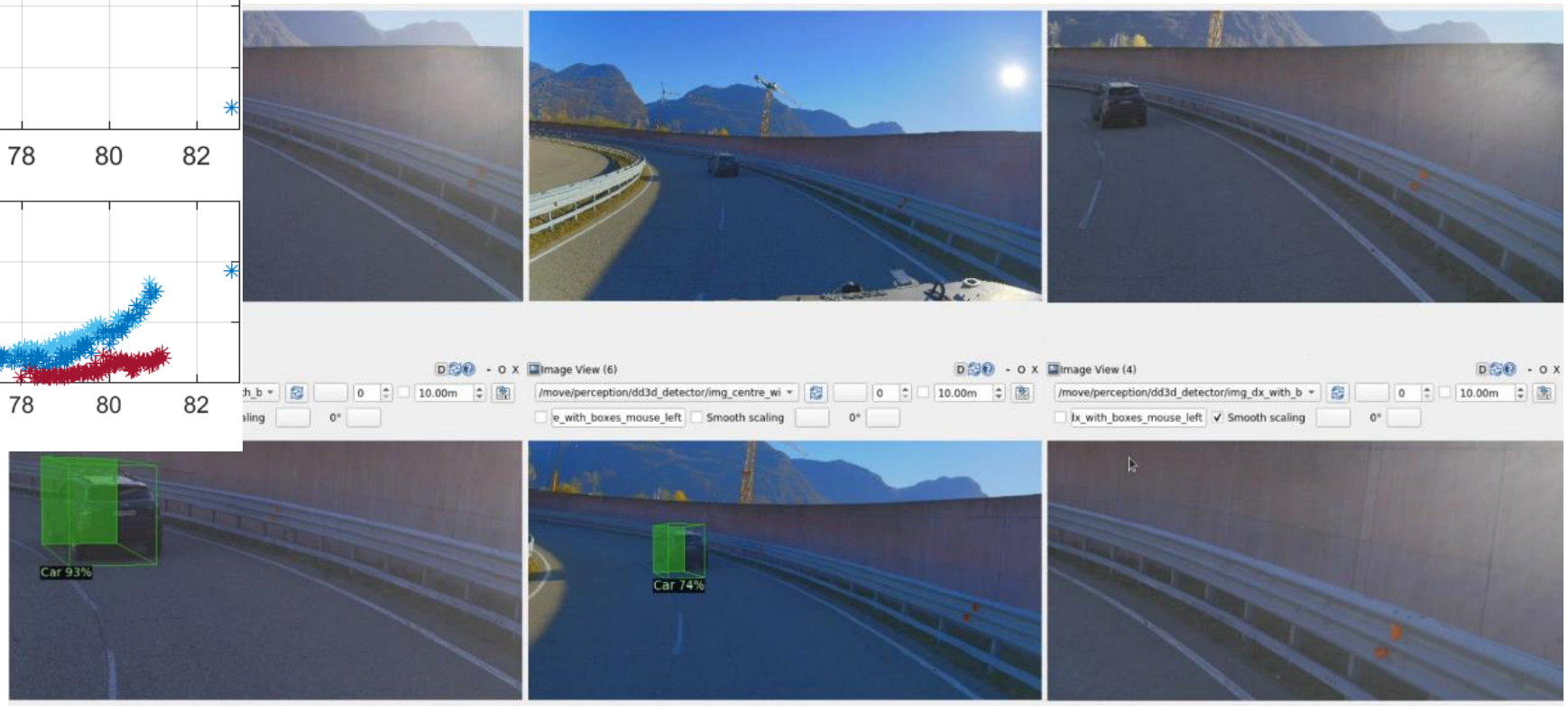
Target Tracking Output



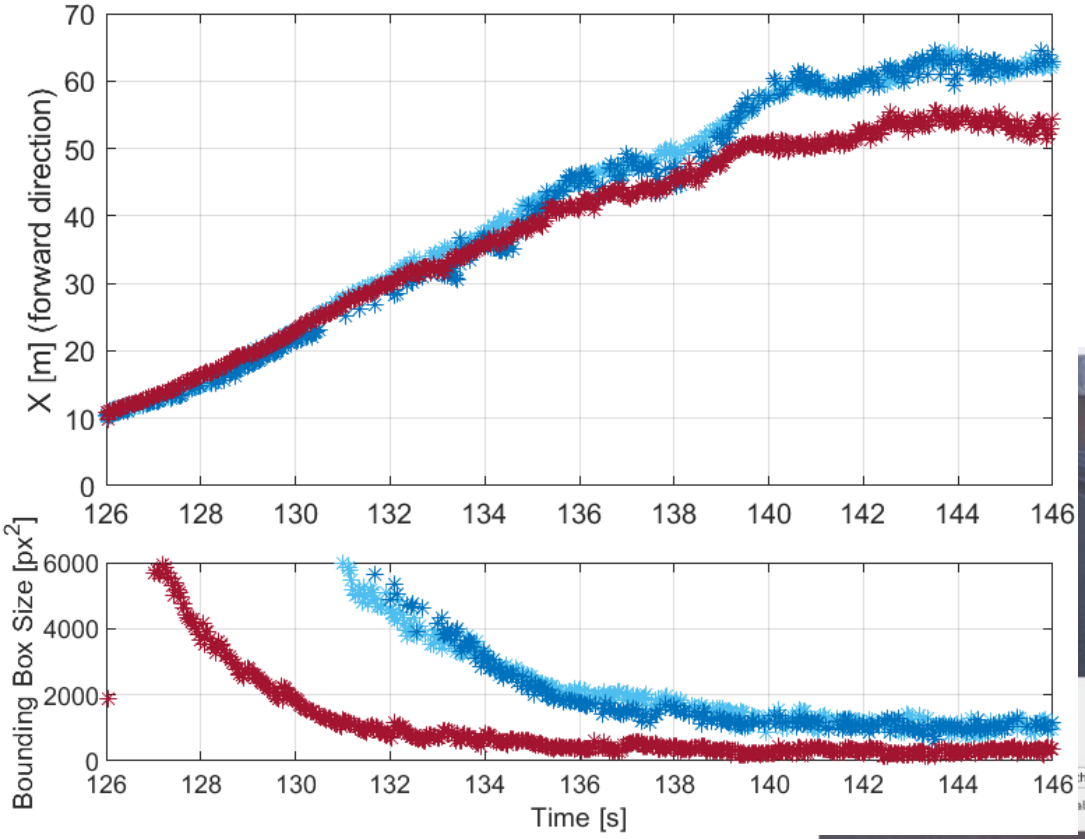
1 – Out of Range



- The 120° camera is not able to detect the target at a distance superior to 50 meters



2 – Inconsistent Range



- The 120° saturates the detected distance.
- There are too little pixels in the Bounding Box to correctly estimate the distance from the target

